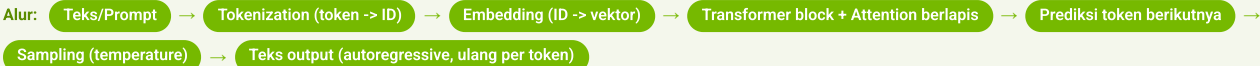


LLM Fundamentals

Mengenal Large Language Models (LLM) berbasis Transformer: dari cara kerjanya (token → embedding → attention → prediksi kata) sampai cara memakai, memproduksi, mengadaptasi, dan mengukurnya. Lima notebook (BANGUN → PAKAI → PRODUKSIKAN → ADAPT → UKUR), semua jalan di Colab Tesla T4 16GB gratis.



NLP vs LLM

- NLP = bidang luas pemrosesan bahasa
- LLM = Large Language Model, satu jenis model NLP
- Dilatih pada teks masif (miliaran kata) + arsitektur transformer
- 1 LLM mengerjakan banyak tugas tanpa dilatih ulang per tugas

► memahami posisi LLM di dalam NLP

Skala & Evolusi

- Evolusi: n-gram -> RNN/LSTM -> Transformer (2017)
- Transformer proses kata paralel via attention, bukan satu per satu (RNN)
- Makin banyak parameter -> makin mampu (sampai titik tertentu)
- Data, arsitektur & fine-tuning sama pentingnya; BERT 110M -> GPT-3 175B

Tokenization

- Model baca token, bukan huruf/kata utuh
- Token sering subword (potongan kata); tiap token -> ID angka
- Kata umum = 1 token; kata jarang dipecah jadi beberapa
- Semua teks diubah jadi token ID dulu

► langkah pertama mengubah teks jadi angka

Bagaimana Transformer Bekerja

- Bayangkan rapat: tiap kata adalah satu peserta yang dengar semua peserta lain
- Langkah: teks -> token -> embedding (vektor) -> attention berlapis -> tebak token berikutnya
- Tiap lapisan = ronde diskusi: tiap kata memperbarui "pemahamannya" dari kata sekitar
- Setelah banyak ronde, model menebak kata paling masuk akal sebagai lanjutan

► intuisi inti cara kerja model

Embedding = Cara "Memahami"

- Tiap kata diubah jadi daftar angka (embedding/vektor)
- Makna mirip -> vektor mirip ("raja" dekat "ratu", jauh dari "meja")
- "Memahami" = mengenali POLA dari teks masif + memakai KONTEKS lewat attention
- Ini statistik/pola, BUKAN kesadaran atau pikiran seperti manusia

► demistifikasi: kenapa model seolah paham bahasa

Attention: Q, K, V, O

- Analogi diskusi kelompok untuk tiap kata:
- Q (Query) = pertanyaan kata: "kata mana yang relevan untukku?"
- K (Key) = name-tag/label keahlian tiap kata
- V (Value) = isi informasi tiap kata
- Attention = cocokkan Q-ku ke semua K, ambil campuran-berbobot dari V
- O (Output proj.) = merangkum kembali info yang terkumpul

```
skor = Q · KT / √d
bobot = softmax(skor)
hasil = bobot · V # lalu lewat O
```

► jantung transformer, dijelaskan tanpa rumus berat

Generation & Sampling

- LLM autoregressive: prediksi 1 token lalu ulang
- Token baru jadi input token selanjutnya
- temperature rendah = fokus; tinggi = kreatif/acak
- top_p & repetition_penalty bantu hasil rapi & tidak berulang

```
model.generate(**inputs, do_sample=True,
               temperature=0.7, top_p=0.9,
               repetition_penalty=1.1, max_new_tokens=200)
```

► atur keseimbangan fokus vs kreativitas output

① Transformer dari Nol (BANGUN)

- PyTorch murni; char tokenizer (per huruf)
- Rakit self-attention: Q·K^T/√d -> softmax -> jumlah-berbobot V
- Causal mask (tak boleh lihat masa depan) + multi-head + LayerNorm/GELU/residual
- Susun TinyGPT (~0.8M params), latih di korpus Indonesia kecil, lalu generate

```
scores = Q @ K.transpose(-2,-1) / d**0.5
scores = scores.masked_fill(mask==0, -inf)
out = softmax(scores) @ V
```

► paham transformer dengan membangunnya sendiri

② Pakai Model Instruct (PAKAI)

- Qwen2.5-3B-Instruct (fp16); model siap-jawab
- Chat template + system prompt (apply_chat_template, format ChatML)
- Prompt engineering: zero-shot & few-shot (beri contoh di prompt)
- Klasifikasi: bert-tiny (head mentah -> LABEL_0/1 ~0.5) vs distilbert-sst2 (terlatih) + zero-shot bart-large-mnli (label bebas); chatbot Gradio

```
msgs=[{"role":"system","content":"..."},
       {"role":"user","content":"Halo"}]
inputs=tok.apply_chat_template(msgs,
                               add_generation_prompt=True, return_dict=True,
                               return_tensors="pt").to(model.device)
model.generate(**inputs)
```

► cara benar memakai LLM chat siap pakai

③ Quantization 4-bit (PRODUKSIKAN)

- Mistral-7B fp16 ~16GB -> OOM (tak muat) di T4
- Quantization 4-bit NF4: kecilkan bobot -> muat ~5.5GB
- Di T4 (compute 7.5) WAJIB compute_dtype float16; bf16 CRASH
- Profil memori GPU; batching (padding_side='left'); streaming token (TextStreamer)

```
bnb = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16) # bf16 crash!
AutoModelForCausalLM.from_pretrained(
    name, quantization_config=bnb,
    device_map="auto")
```

► jalankan model 7B di GPU gratis 16GB

④ Fine-Tuning LoRA/QLoRA (ADAPT)

- Model raksasa = miliaran "kenop" (weights); latih ulang semua terlalu mahal
- LoRA: BEKUKAN model besar, tempel matriks kecil (low-rank) yang dilatih — analogi: jangan tulis ulang buku, cukup tempel sticky-note/errata; hanya ~1.2% params dilatih
- QLoRA: kecilkan dulu ke 4-bit (muat di T4), BARU tempel sticky-note LoRA — analogi: buku edisi saku 4-bit + sticky-note
- Demo: Qwen2.5-1.5B + Trainer, ajari fakta Navasena, ~2 menit; banding base vs hasil via disable_adapter()

```
LoraConfig(r=8, lora_alpha=16,
           target_modules=["q_proj","v_proj"])
# base 4-bit dibekukan, hanya adapter dilatih
```

► ajari model baru tanpa GPU mahal

⑤ Evaluasi (UKUR)

- Metrik teks: ROUGE/BLEU (cocok kata), BERTScore (cocok makna), perplexity
- Uji statistik: paired t-test, Cohen's d, 95% CI
- LLM-as-judge: minta LLM lain menilai (Qwen3B lokal / cloud NVIDIA NIM)
- Pelajaran: metrik bisa unggulkan model lebih besar, tapi p>0.05 = belum signifikan secara statistik

```
t, p = ttest_rel(skor_A, skor_B)
# p < 0.05 -> beda benar-benar nyata
```

► buktikan model mana lebih baik secara jujur

Quick-Ref Library & Model (5 Notebook)

Notebook	Library/Model	Tujuan
00 · BANGUN — Transformer dari Nol	PyTorch (TinyGPT ~0.8M)	Rakit self-attention & latih GPT mini sendiri
01 · PAKAI — Model Instruct	Qwen2.5-3B-Instruct (fp16); distilbert-sst2; bart-large-mnli; Gradio	Chat template, prompt eng, klasifikasi, chatbot
02 · PRODUKSIKAN — Quantization	Mistral-7B-Instruct-v0.3 + BitsAndBytes 4-bit NF4	Muat 7B ~5.5GB di T4, batching & streaming
03 · ADAPT — Fine-Tuning	QLoRA: Qwen2.5-1.5B + LoRA adapter + Trainer	Latih ~1.2% params, ajari fakta baru ~2 menit
04 · UKUR — Evaluasi	ROUGE/BLEU/BERTScore, perplexity, NVIDIA NIM judge	Metrik + uji statistik + LLM-as-judge

Istilah Kunci

- LLM** — Large Language Model: model NLP raksasa berbasis transformer, dilatih pada teks masif.
- Token** — Potongan teks (sering subword) yang dipetakan ke ID angka untuk dibaca model.
- Embedding** — Vektor angka mewakili token; kata mirip makna -> vektor mirip.
- Attention** — Mekanisme model menimbang kata mana yang paling relevan saat memproses kalimat.
- Q / K / V** — Query (pertanyaan kata), Key (label keahlian kata), Value (isi info kata); attention mencocokkan Q ke semua K lalu ambil campuran-berbobot V.
- Transformer** — Arsitektur (2017) yang memproses kata paralel via attention, dasar LLM modern.
- Autoregressive** — Cara generasi: prediksi 1 token, jadikan input berikutnya, ulang terus.
- Temperature** — Parameter sampling: rendah = fokus/deterministik, tinggi = kreatif/acak.
- Chat template** — Format pesan baku (system/user/assistant, mis. ChatML) lewat apply_chat_template agar model instruct paham peran.
- Quantization** — Kecilkan presisi bobot (mis. 16-bit -> 4-bit NF4) agar model muat di GPU kecil & lebih hemat memori.
- LoRA** — Bekukan model besar, tambah matriks kecil (low-rank) yang dilatih (~1%); analogi sticky-note, bukan tulis ulang buku.
- QLoRA** — LoRA di atas model 4-bit: kecilkan dulu (quantization) lalu tempel adapter; muat di T4 gratis.
- Perplexity** — Ukuran "kebingungan" model menebak teks; makin rendah makin baik.
- BLEU / ROUGE** — Metrik kemiripan kata output vs jawaban acuan (BLEU presisi, ROUGE recall); BERTScore menilai kemiripan makna.